

Experiment – 1.3.1

Student Name: Damandeep Singh

UID: 24BDA7O313

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 22-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To design and implement a secure authentication system using JWT for user login and session management.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Backend: Express.js, jsonwebtoken, bcryptjs, cors
- Frontend: React 18, Vite, Redux Toolkit, react-redux
- Browser: Google Chrome (for testing and rendering)

3. Objective:

To design and implement a secure authentication system using JSON Web Tokens (JWT) for user login and session management. Passwords must never be stored or compared in plain text; the server should issue a signed, time-limited token on successful login, and the client should persist and revalidate that token across page reloads.

4. Theory:

A JSON Web Token is a compact, digitally signed string consisting of a header, payload, and signature, base64url-encoded and separated by dots. The server signs the token with a secret key (jwt.sign); any later request presents the token in an Authorization: Bearer <token> header, and the server verifies the signature and expiry (jwt.verify) before trusting its payload. Because the payload is only encoded, not encrypted, it should never contain sensitive data — only identifying claims such as user id, username, and role. Passwords are never stored directly; bcrypt hashes them with a per-password salt (bcrypt.hashSync), and



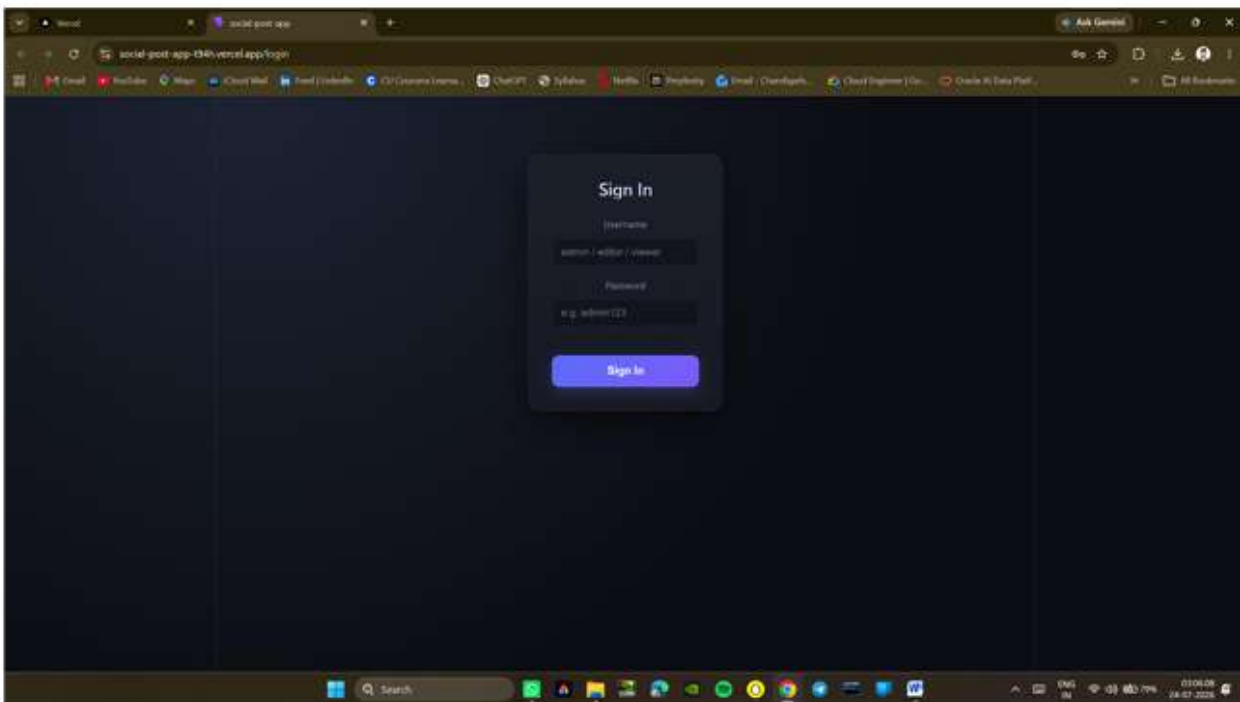
login compares a submitted password against the stored hash (bcrypt.compareSync) rather than comparing plain text.

5. Procedure/Algorithm:

- Step 1: Store users with bcrypt-hashed passwords and an assigned role.
- Step 2: On POST /api/login, look up the user, compare the password hash, and reject with 401 if either fails.
- Step 3: On success, sign a JWT containing { id, username, role } with a 15-minute expiry and return it to the client.
- Step 4: Write an authenticate middleware that extracts the Bearer token, verifies it, and attaches the decoded payload to req.user.
- Step 5: Build an authSlice with a login thunk that calls /api/login and stores the returned token in Redux and localStorage.
- Step 6: Build a checkSession thunk that calls /api/me with the stored token on app load, clearing the session if it's invalid or expired.

6. Code and Output:

```
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
  const user = users.find((u) => u.username === username);
  if (!user || !bcrypt.compareSync(password, user.passwordHash)) {
    return res.status(401).json({ message: 'Invalid username or password' });
  }
  const token = jwt.sign(
    { id: user.id, username: user.username, role: user.role },
    JWT_SECRET,
    { expiresIn: '15m' }
  );
  res.json({ token, user: { id: user.id, username: user.username, role: user.role } });
});
```



7. Conclusion:

A working authentication system was implemented where passwords are hashed with bcrypt and never stored or compared in plain text, and a signed, time-limited JWT is issued on successful login and verified on every subsequent request. The session persists across page reloads via localStorage and is revalidated against the server.

8. Learning Outcomes:

- Learned why passwords must be hashed rather than stored or compared in plain text.
- Understood the structure of a JWT and why its payload must never contain sensitive data.
- Learned to persist and revalidate a session across page reloads using localStorage and a session-check API call.